National University of Computer and Emerging Sciences

Lab Manual 03
Artificial Intelligence Lab

Department of Computer Science

FAST-NU, Lahore, Pakistan

Learning Outcome

Students will learn how to use search algorithms on graphs, including understanding the workings of DFS and BFS. They will also gain knowledge of the pros and cons associated with both DFS and BFS.

Search in AI is the process of navigating from a **starting state** to a **goal state** by **transitioning** through **intermediate states**.

Almost any AI problem can be defined in these terms.

- **State** — A potential outcome of a problem
  - **Transition** — The act of moving between states.
  - **Starting State —** Where to start searching from.
  - **Intermediate State**- The states between the starting state and the goal state that we need to transition to.
  - **Goal State —** The state to stop searching.
  - **Search Space** — A collection of states.

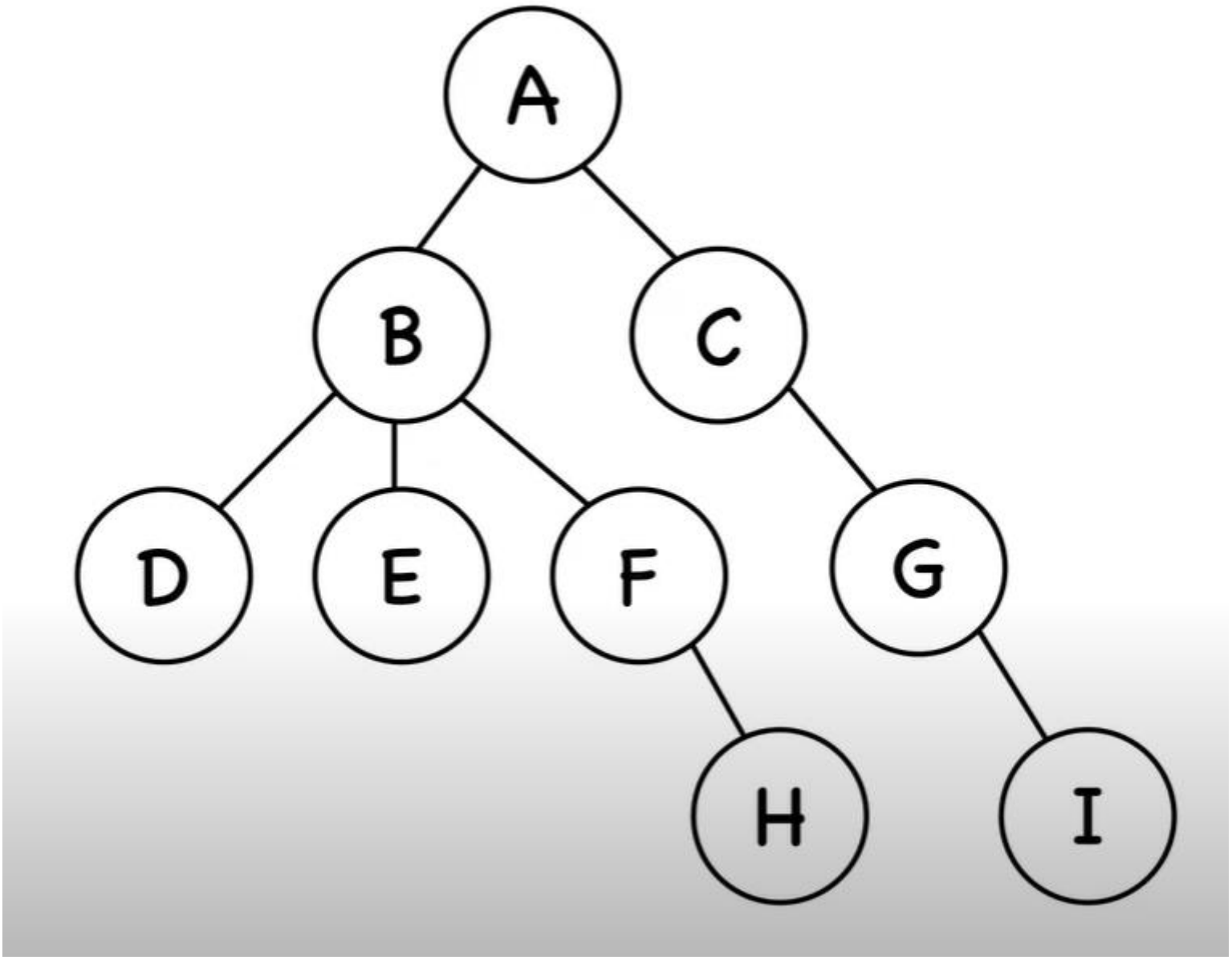
Breadth-first search

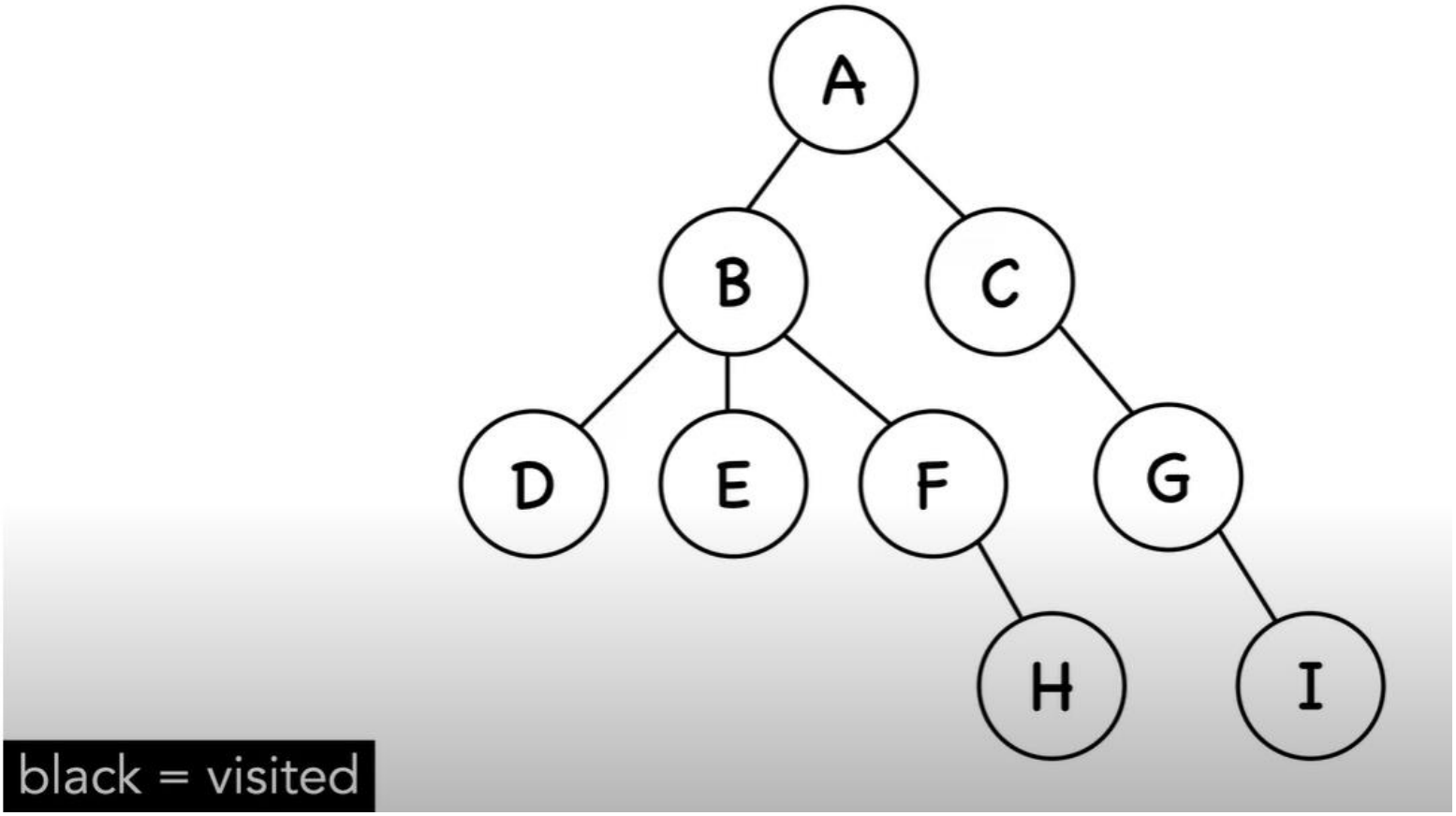
- Breadth-first search is an algorithm for searching a tree data structure for a node that satisfies a given property. It starts at the tree root and explores all nodes at the present depth prior to moving on to the nodes at the next depth level.

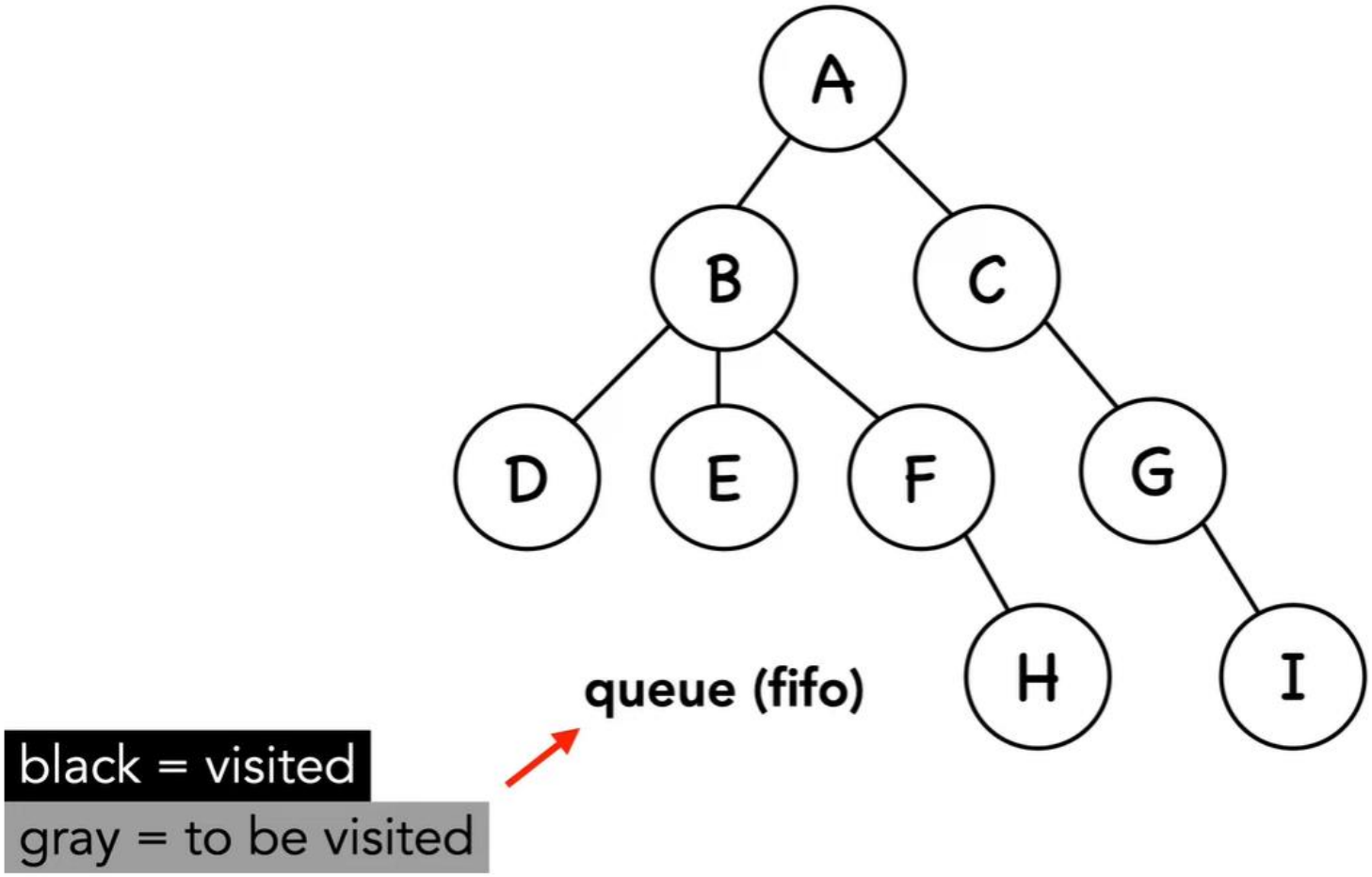
Data Structure use in Breadth-first search

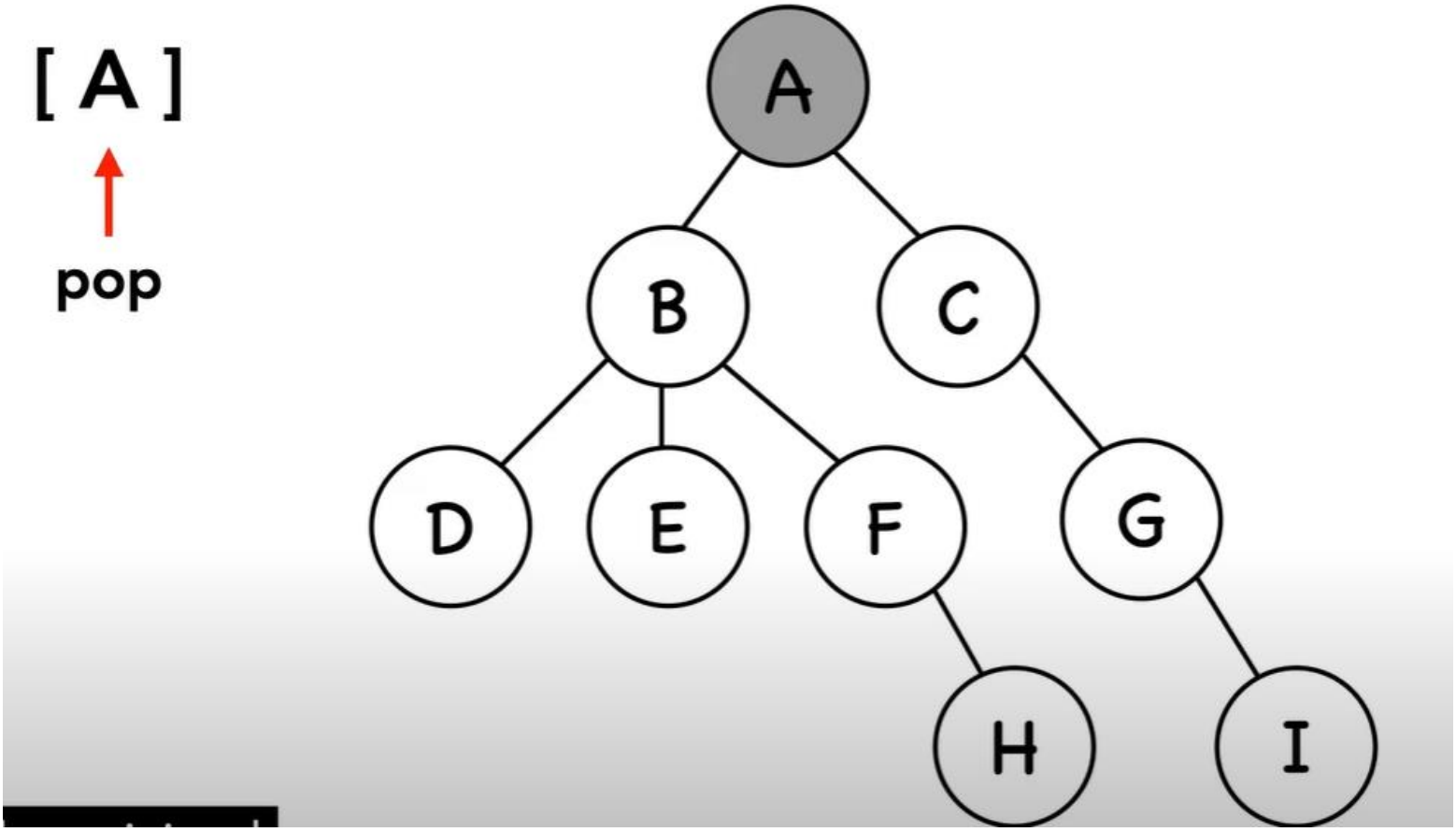
- Important data structure to keep track of vertices is FIFO QUEUE

Breadth-first search Example Demonstration is given below.

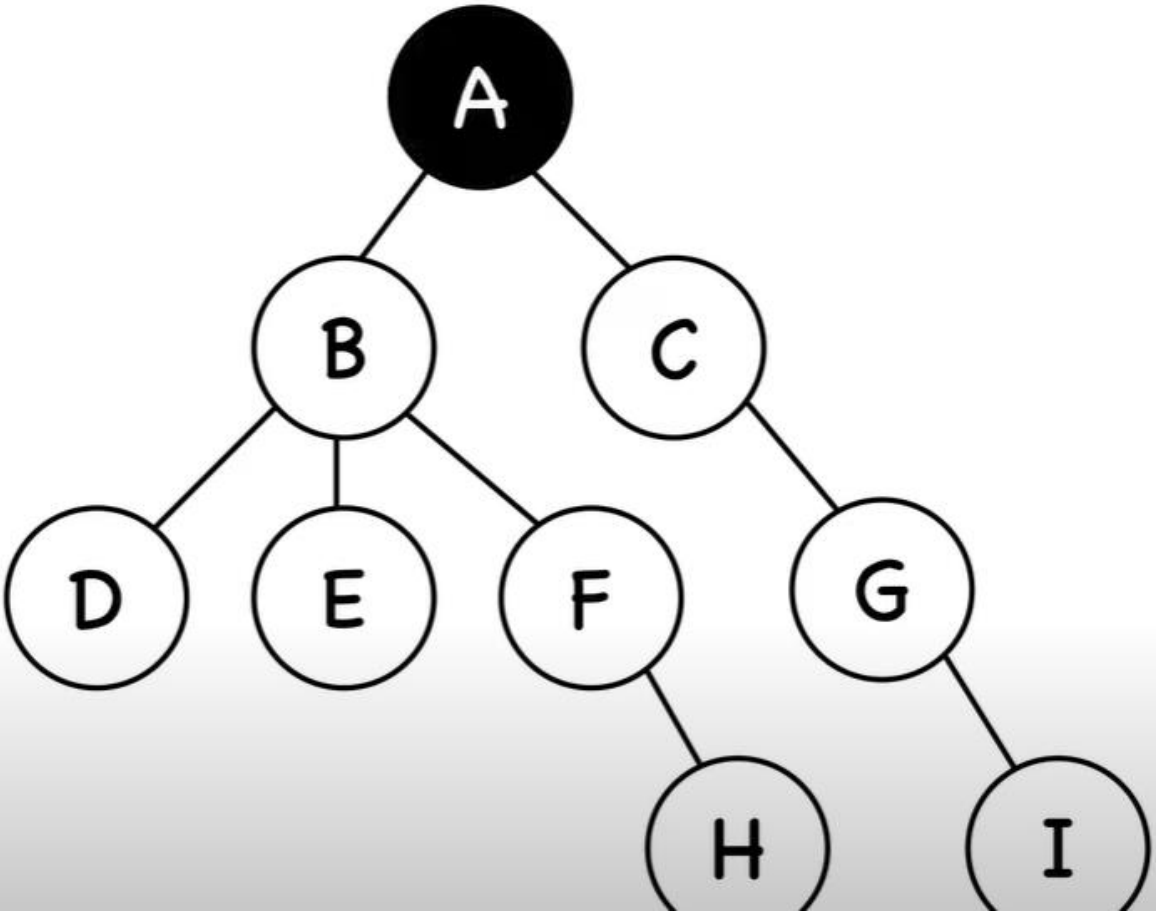

A
B
C
D
E
F
G
H
I


A
B
C
D
E
F
G
H
I
black = visited

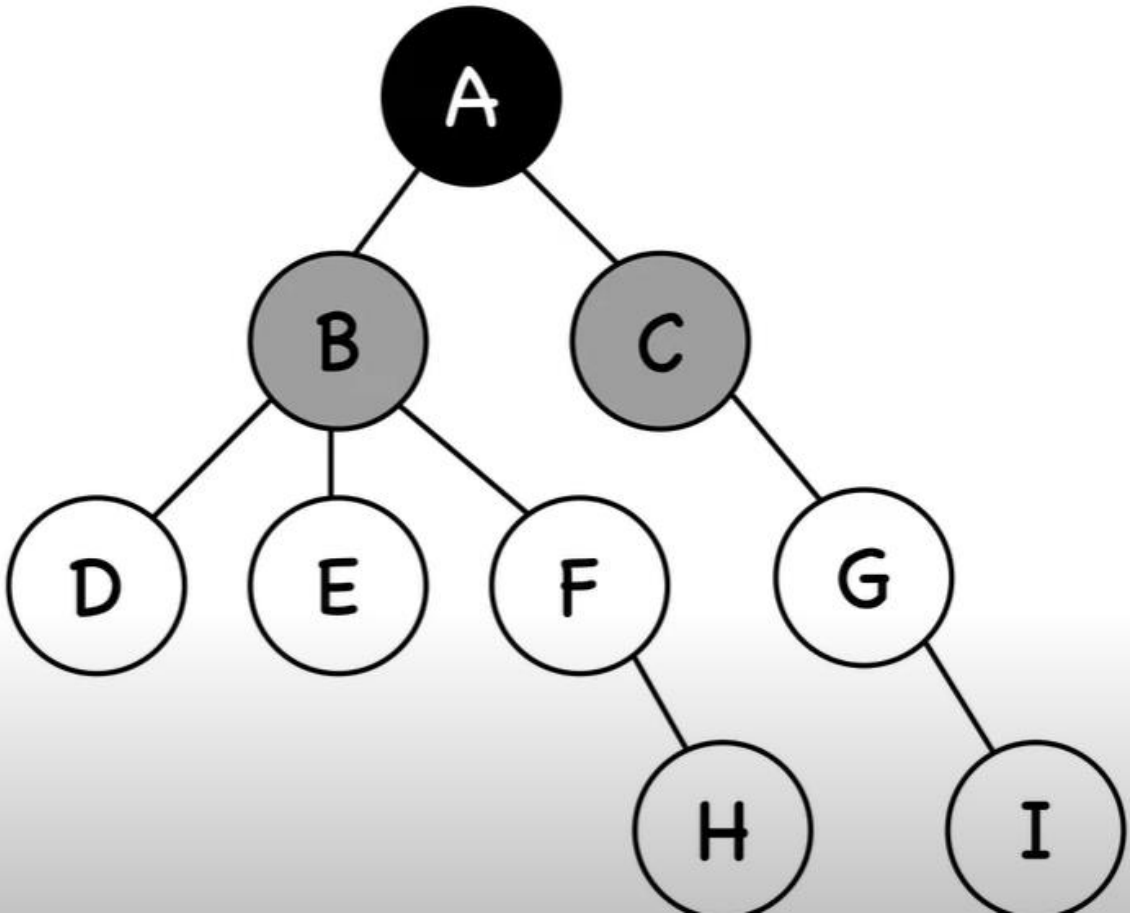

A
B
C
D
E
F
G
H
I
queue (fifo)
black = visited
gray = to be visited


[A]
pop
A
B
C
D
E
F
G
H
I

[ ]

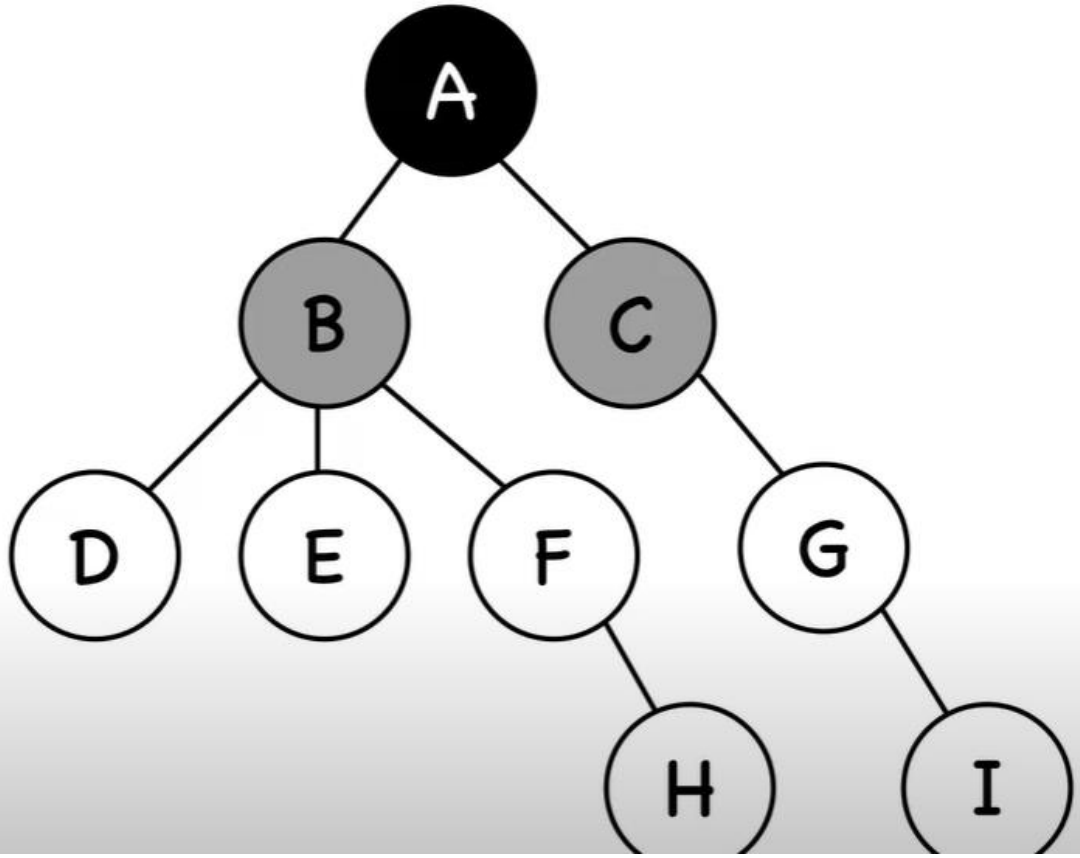

A
B
C
D
E
F
G
H
I

[C, B]

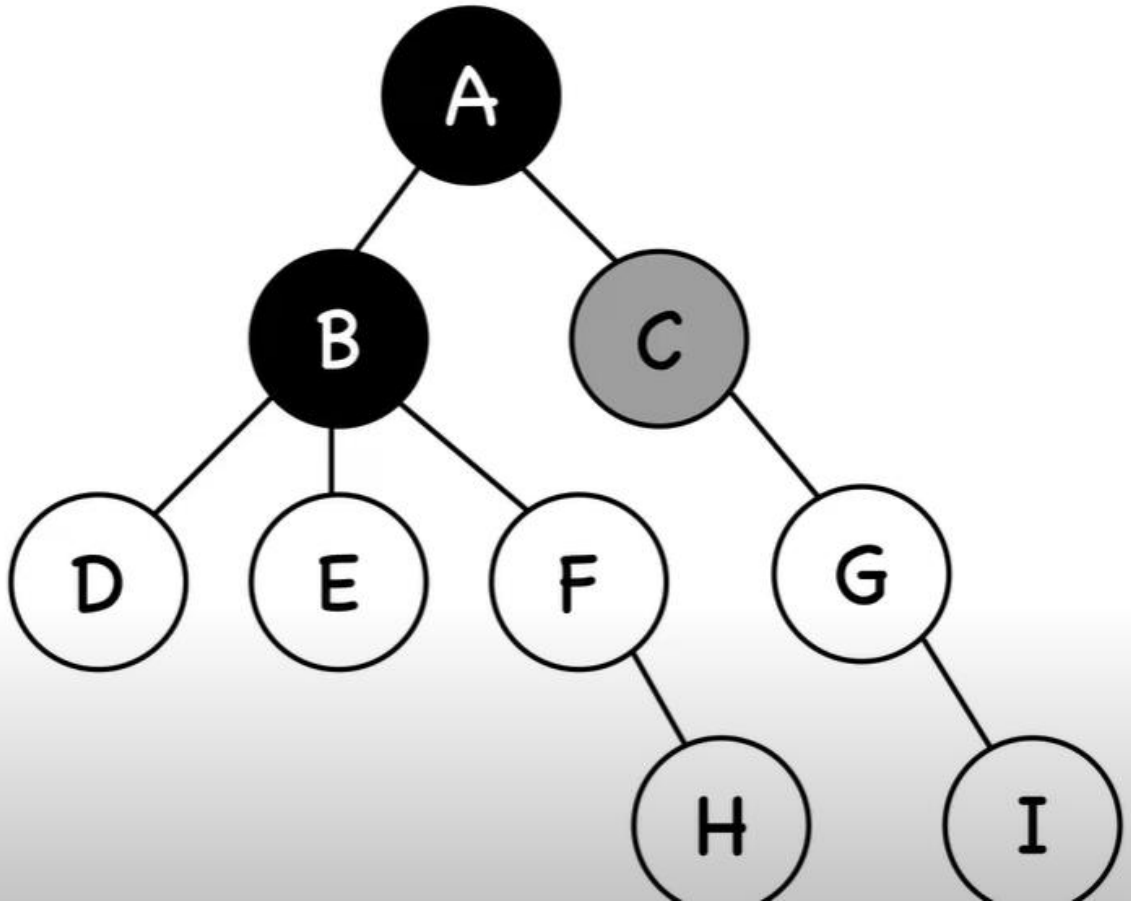


[C, B]

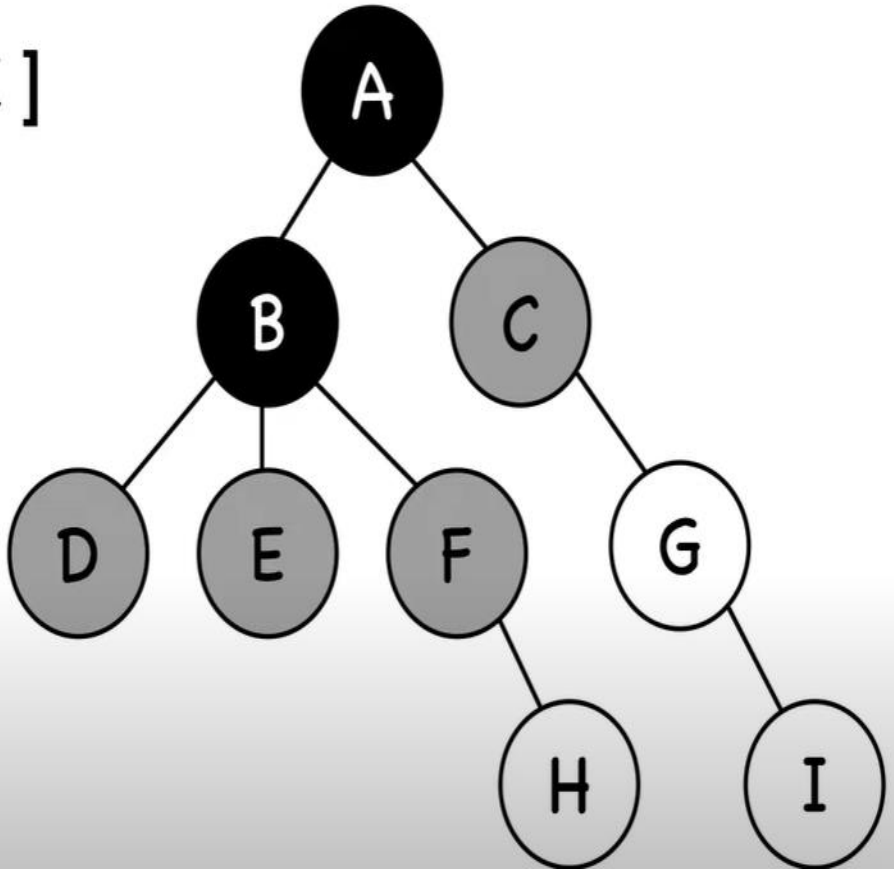
pop

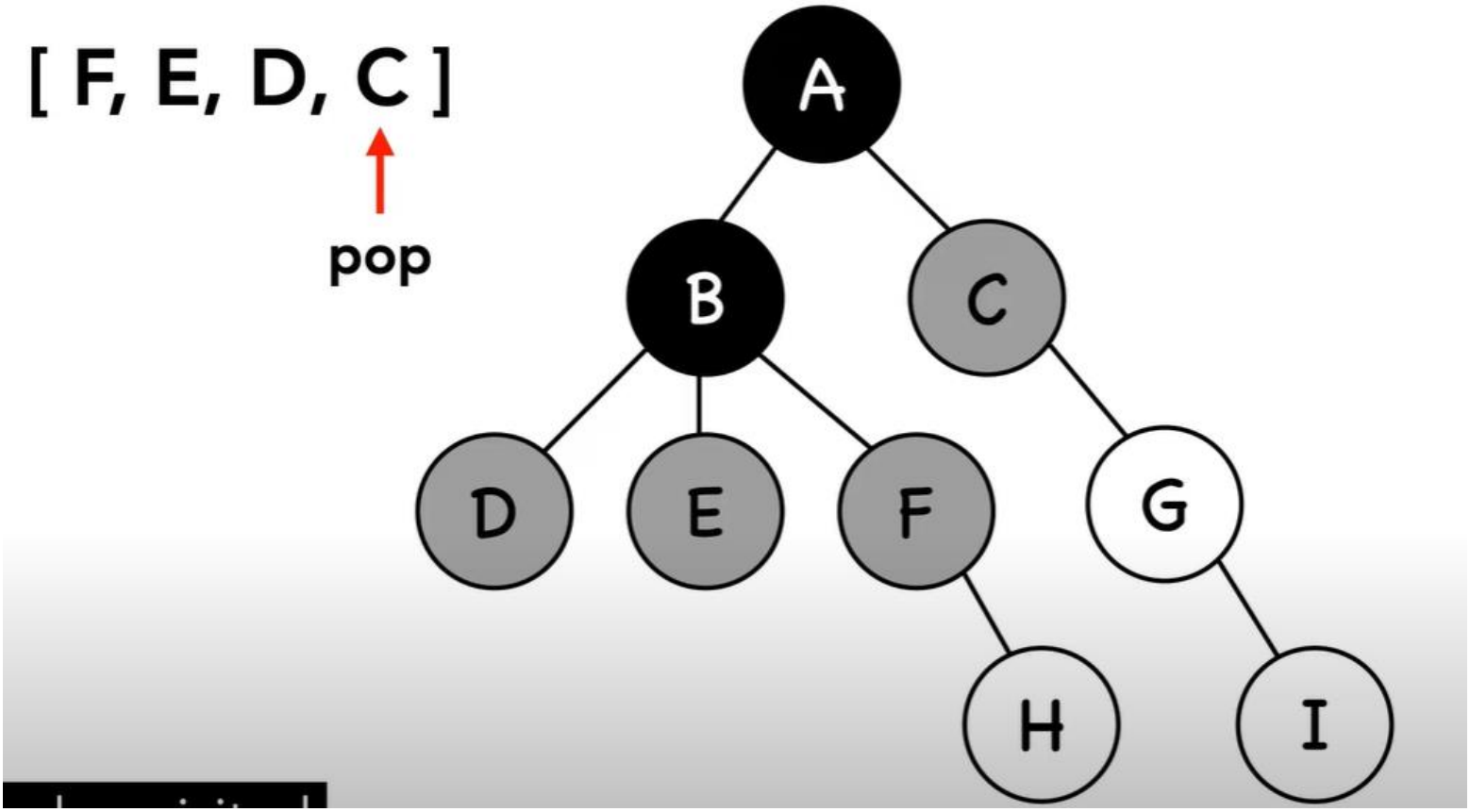


[C]

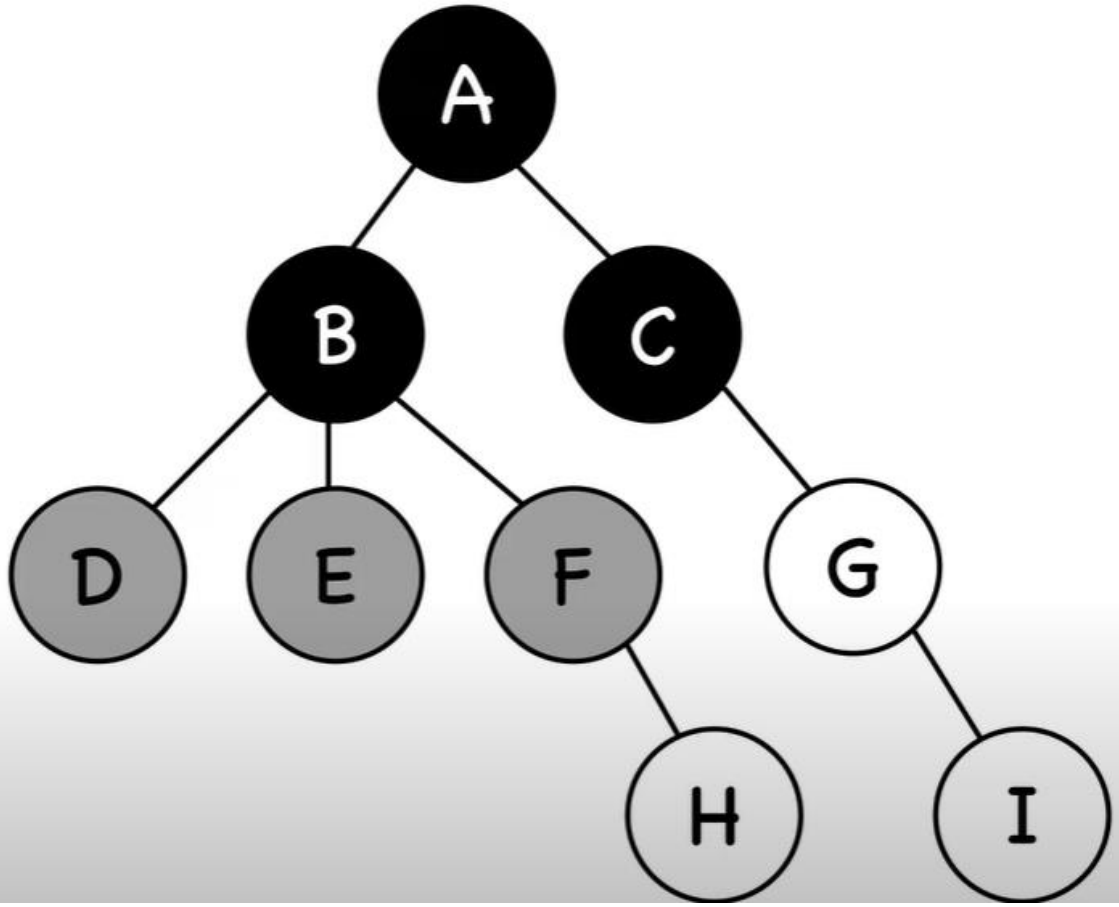


[F, E, D, C]

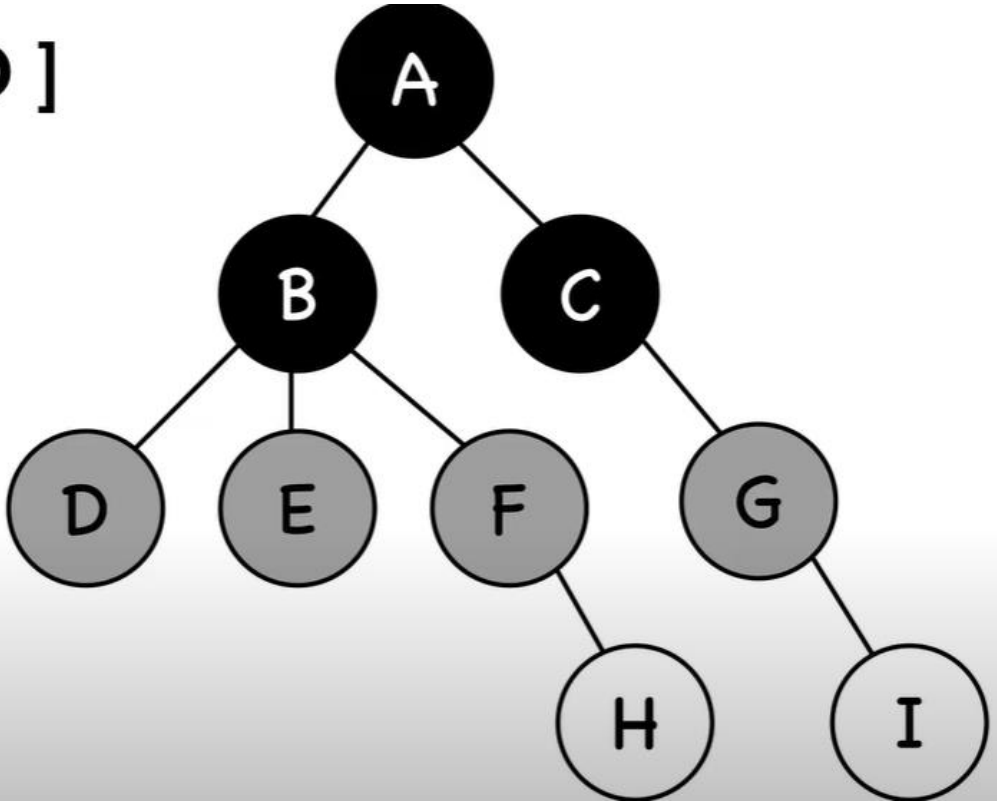



[F, E, D, C]
pop
A
B
C
D
E
F
G
H
I

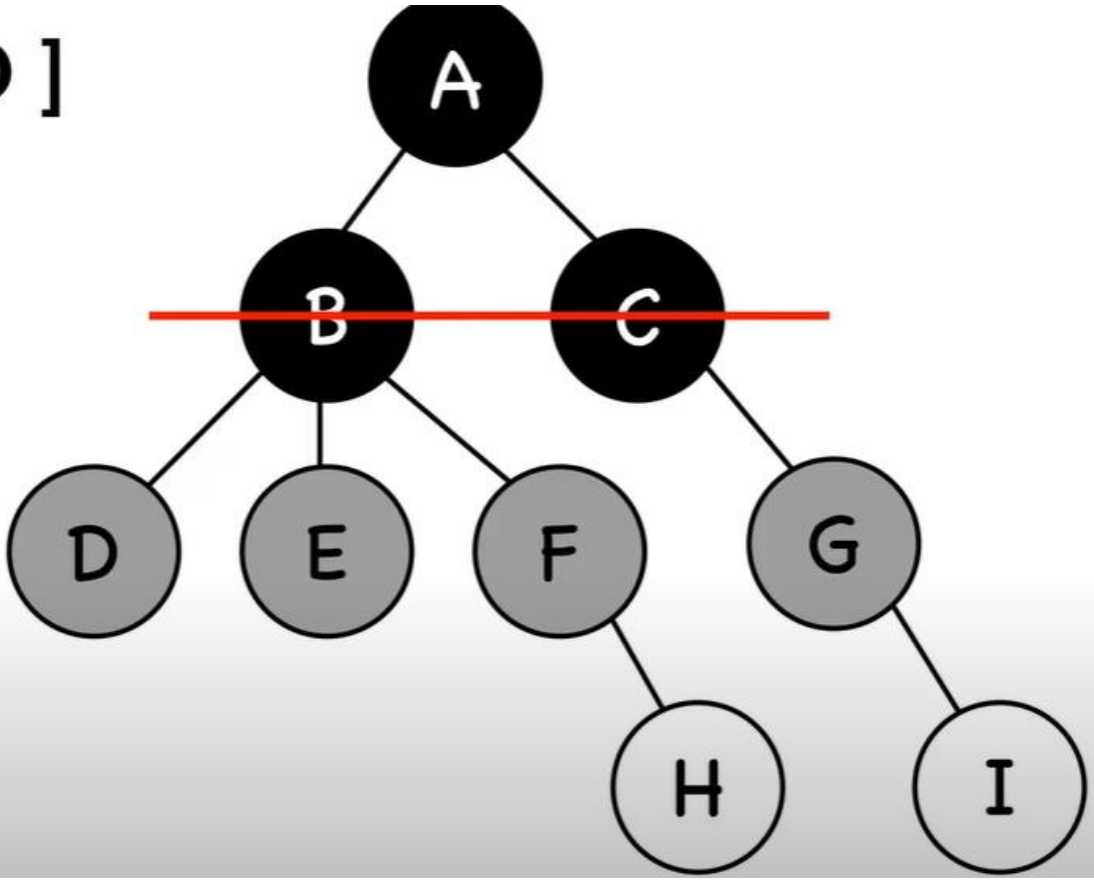
[F, E, D]



[G, F, E, D]

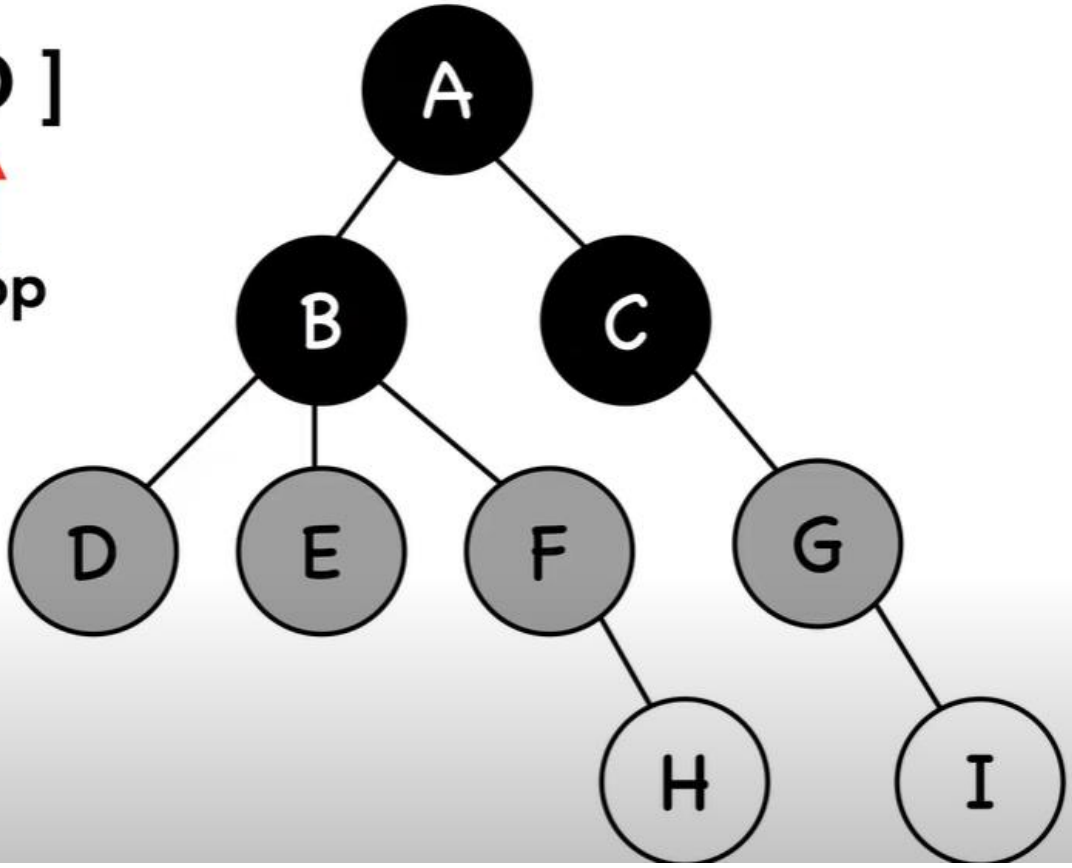


[G, F, E, D]

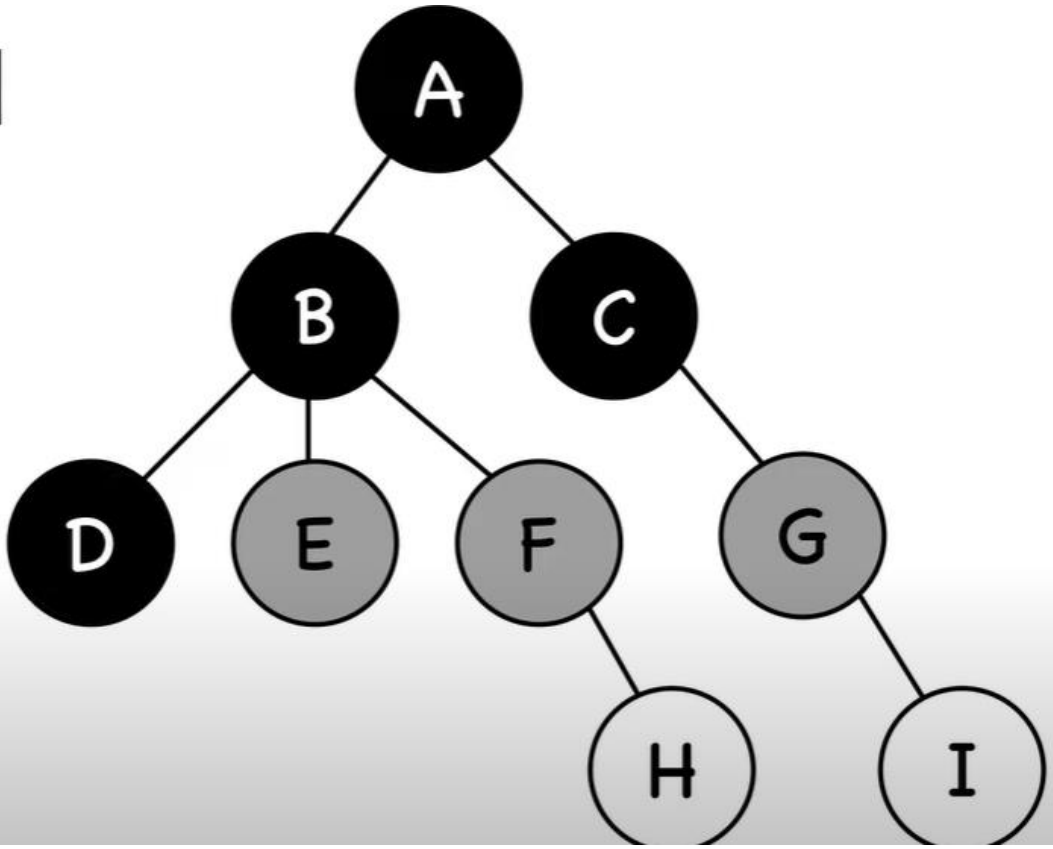


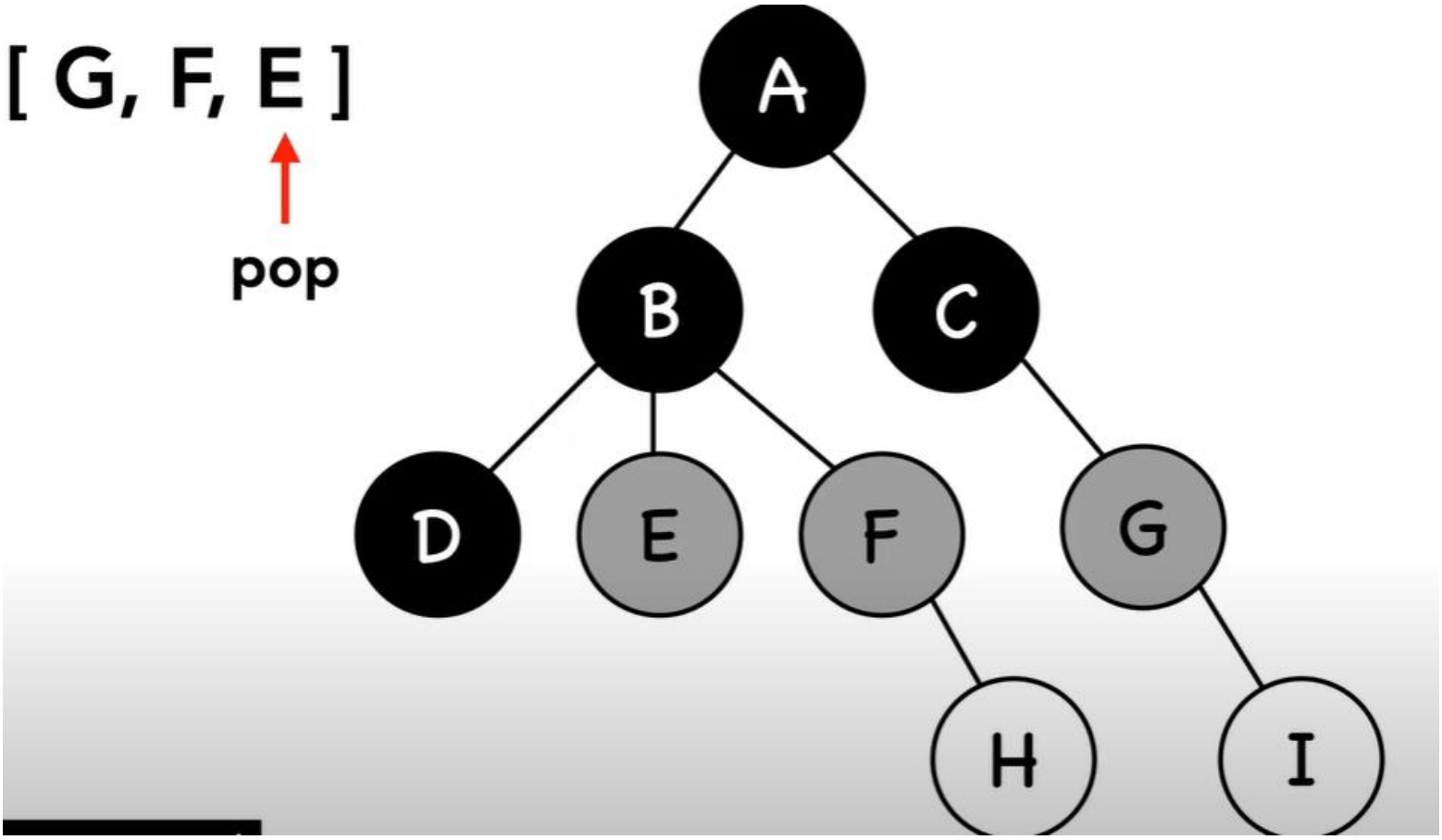
[G, F, E, D]

pop

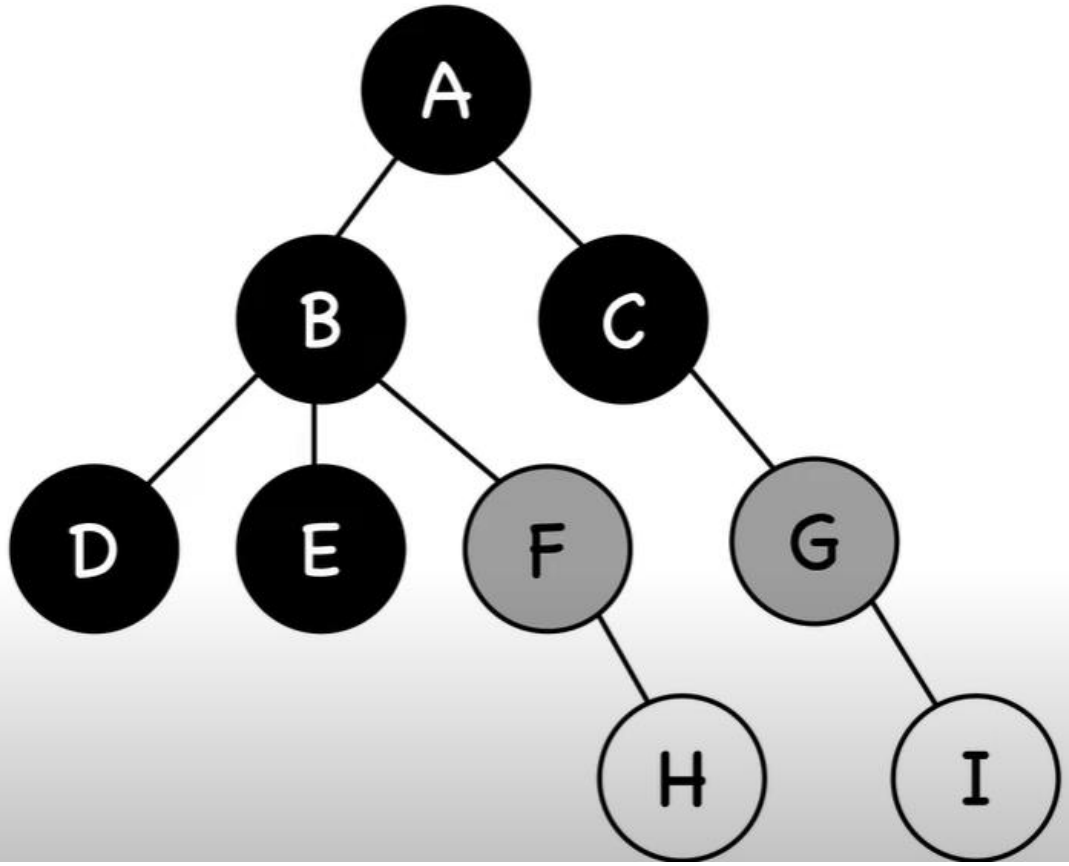


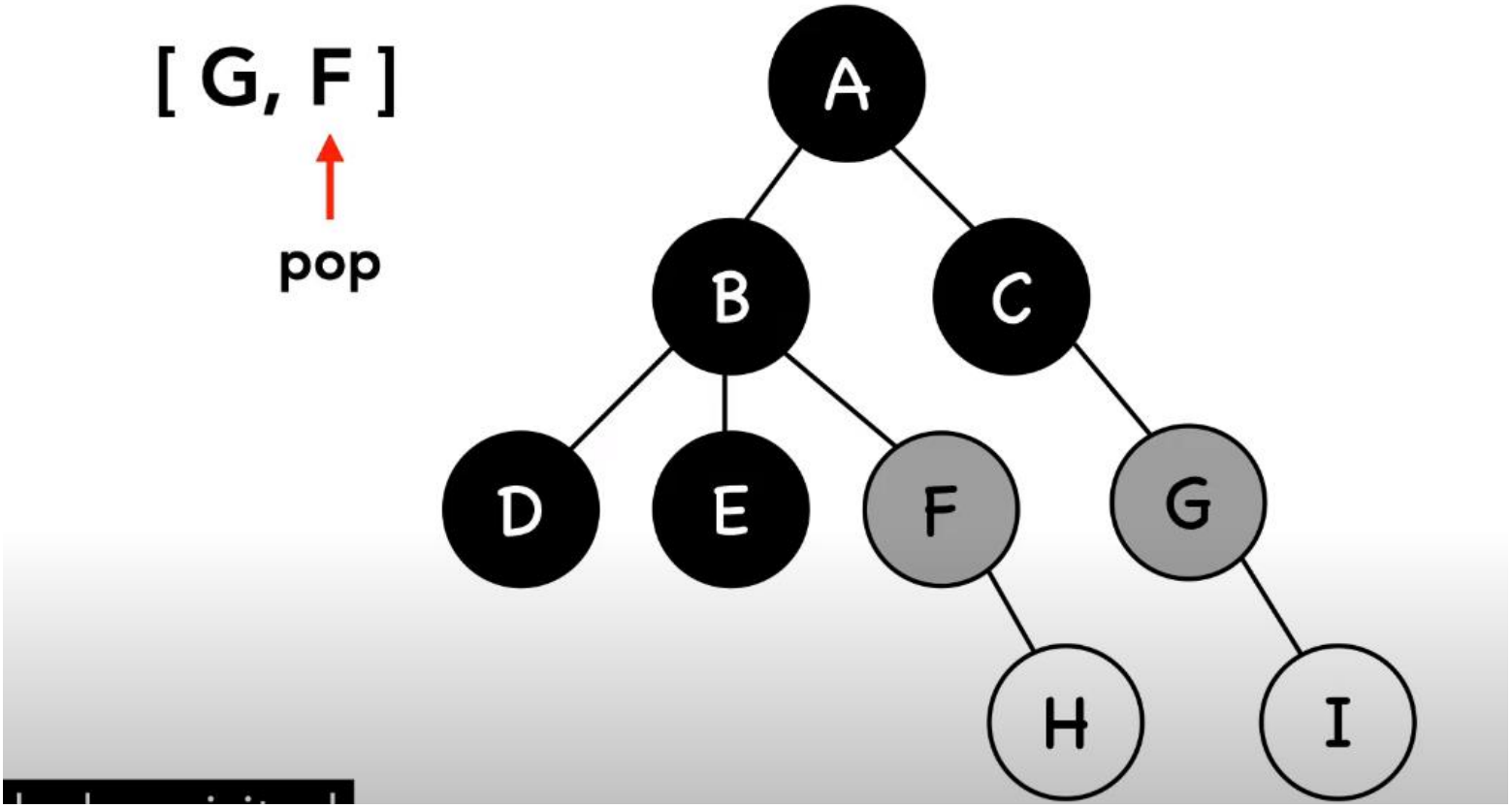
[G, F, E]



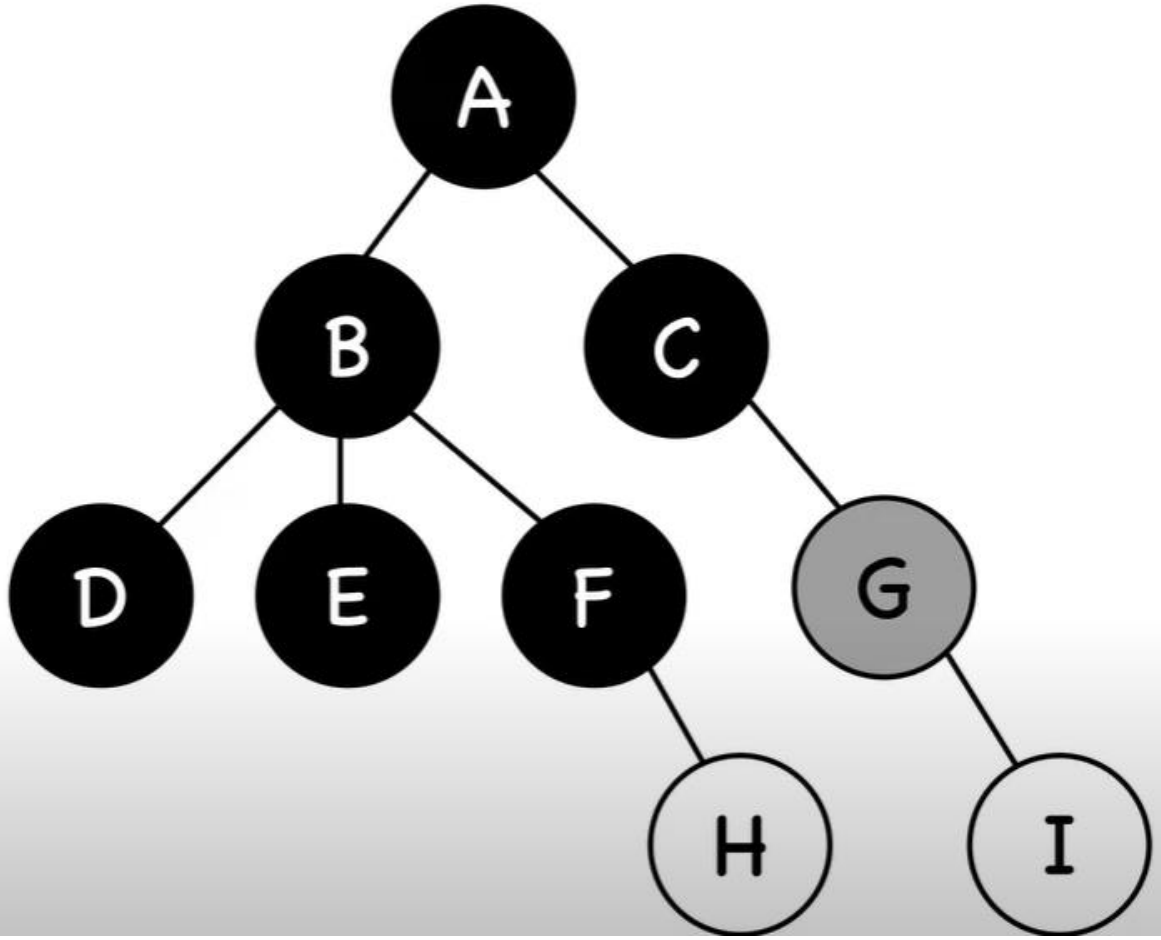

[G, F, E]
pop
A
B
C
D
E
F
G
H
I

[G, F]

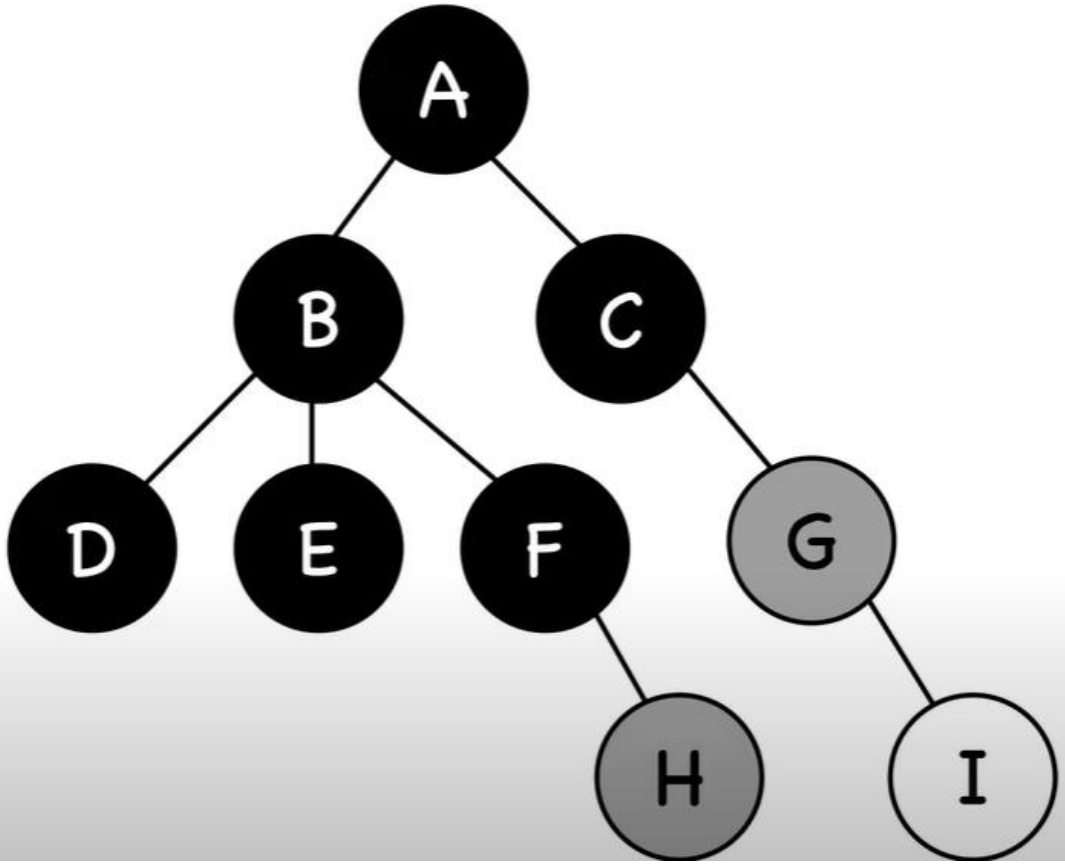



[G, F]
pop
A
B
C
D
E
F
G
H
I

[G]

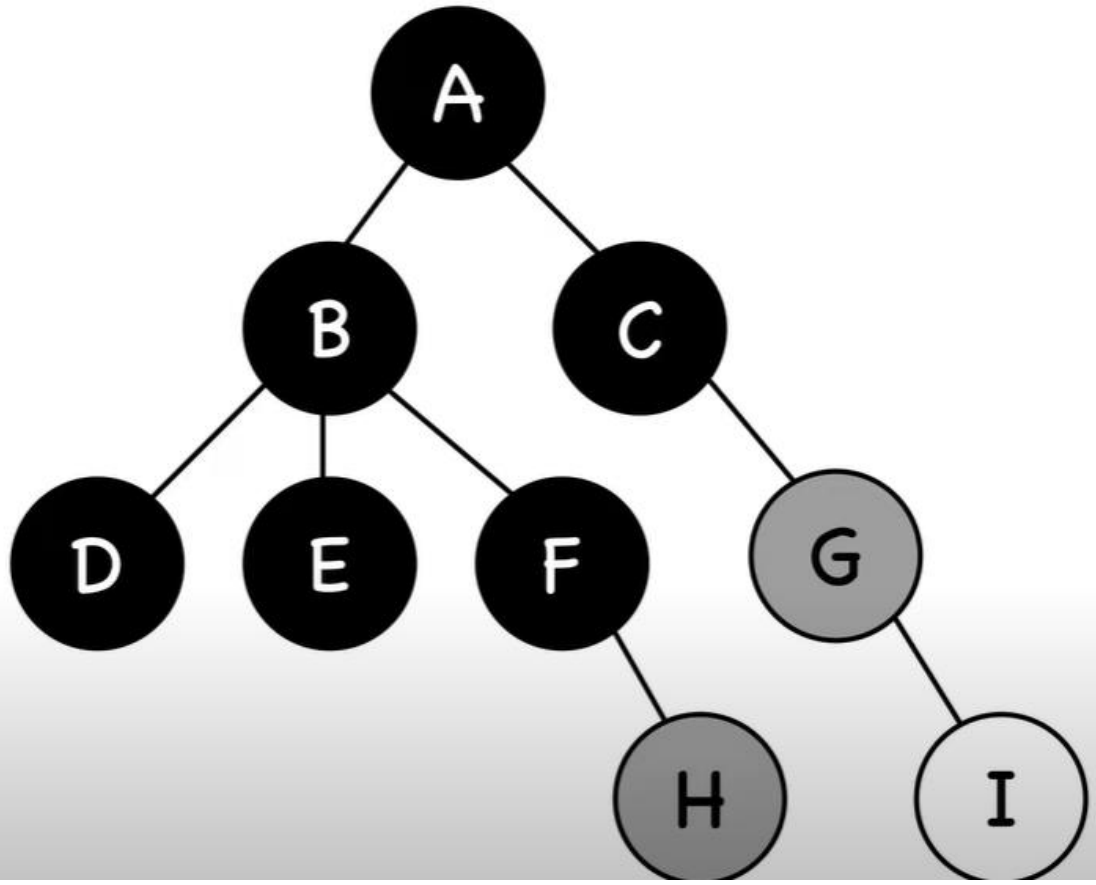


[H, G]

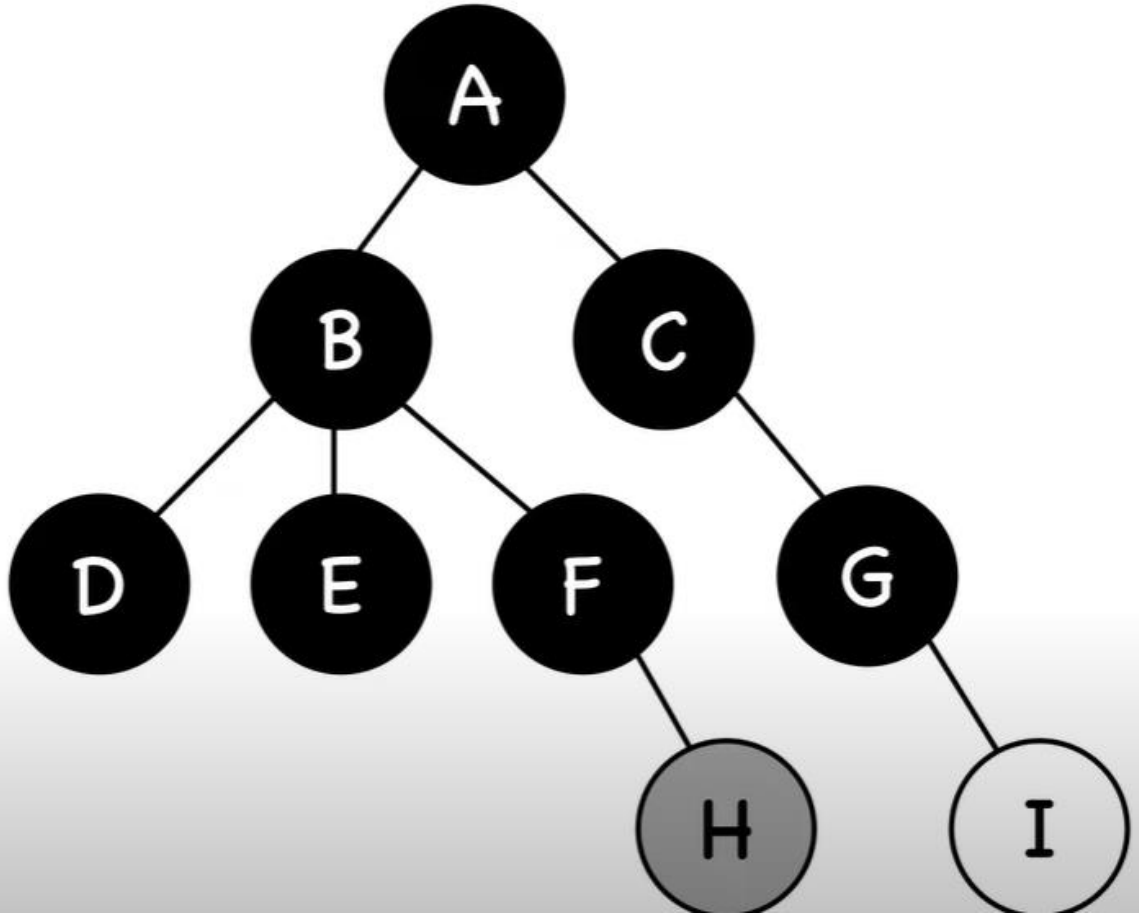


[H, G]

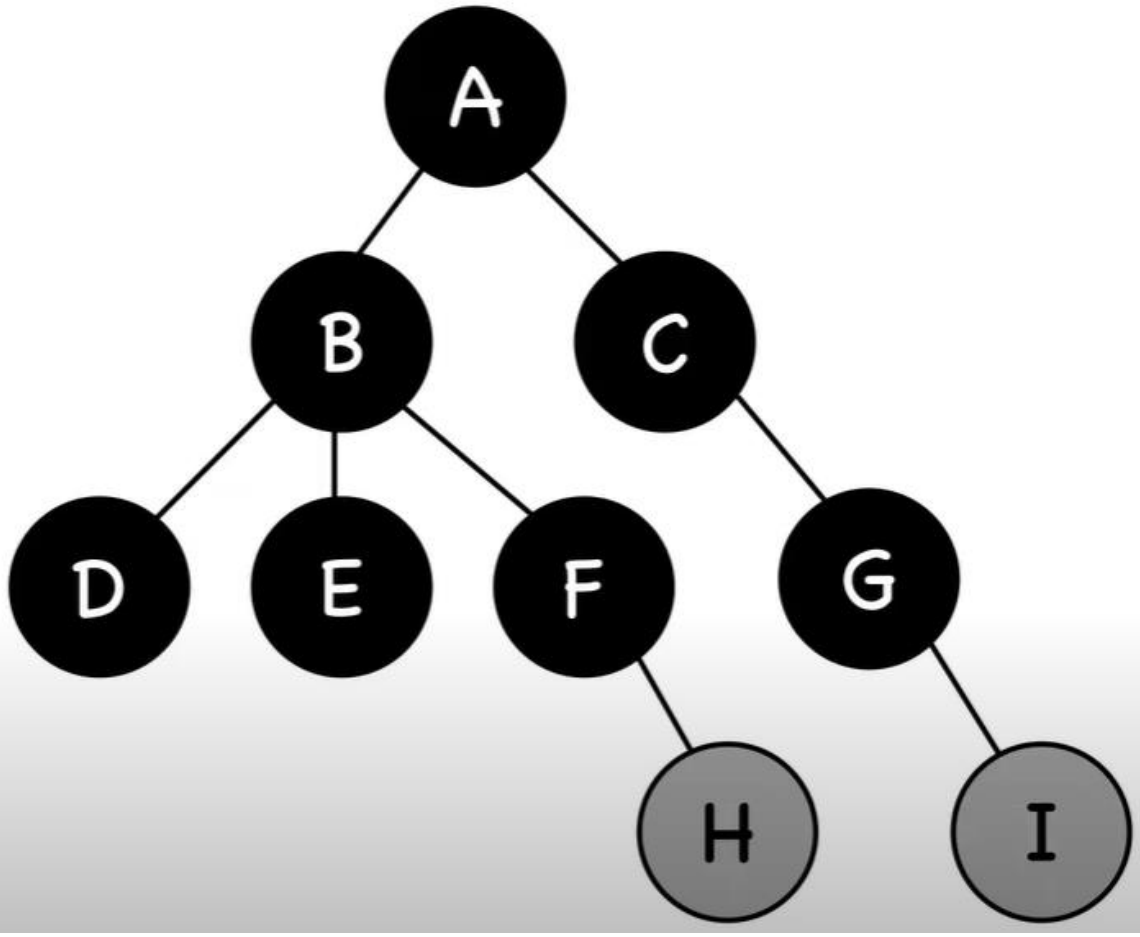
pop

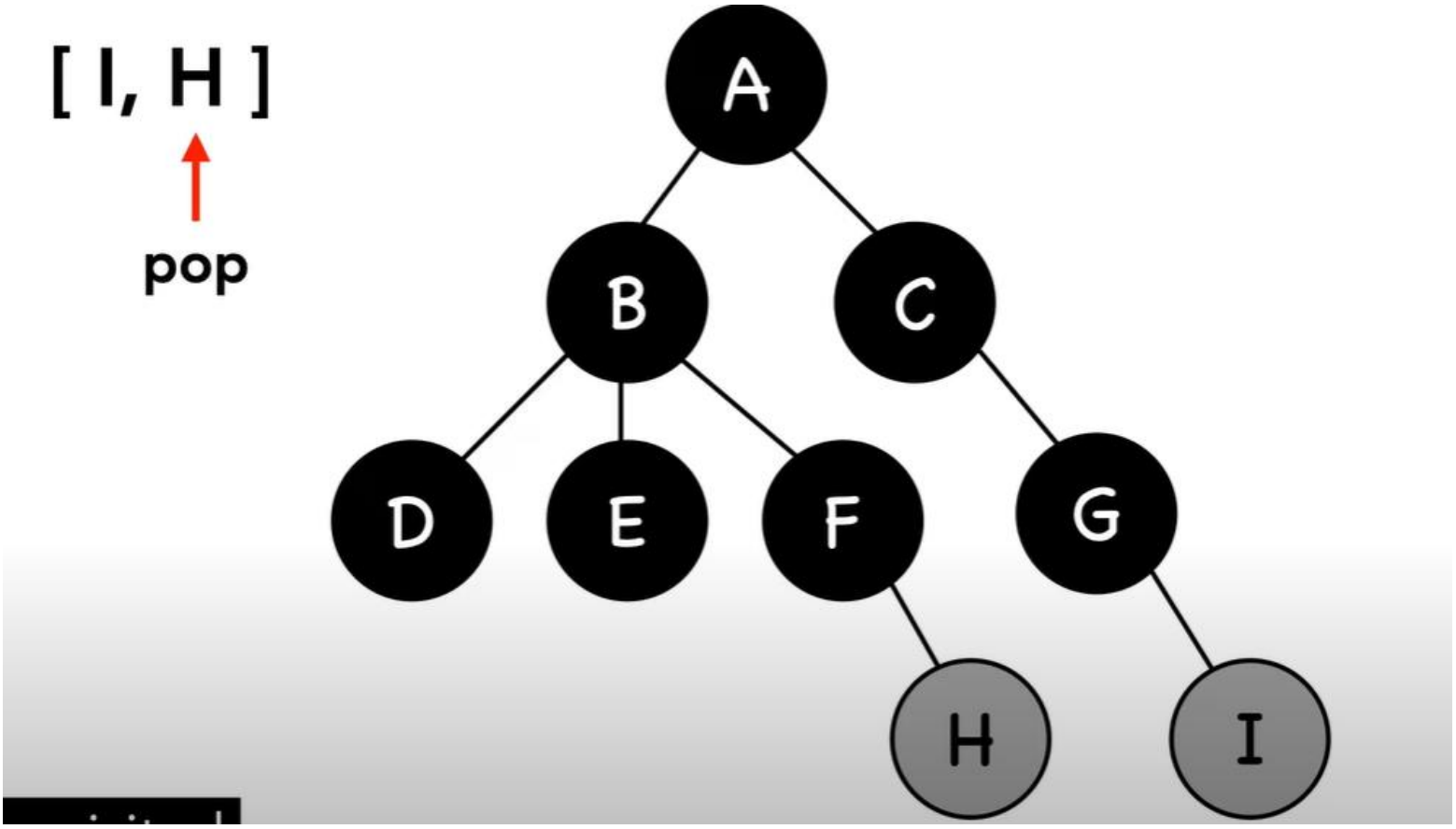


[H]

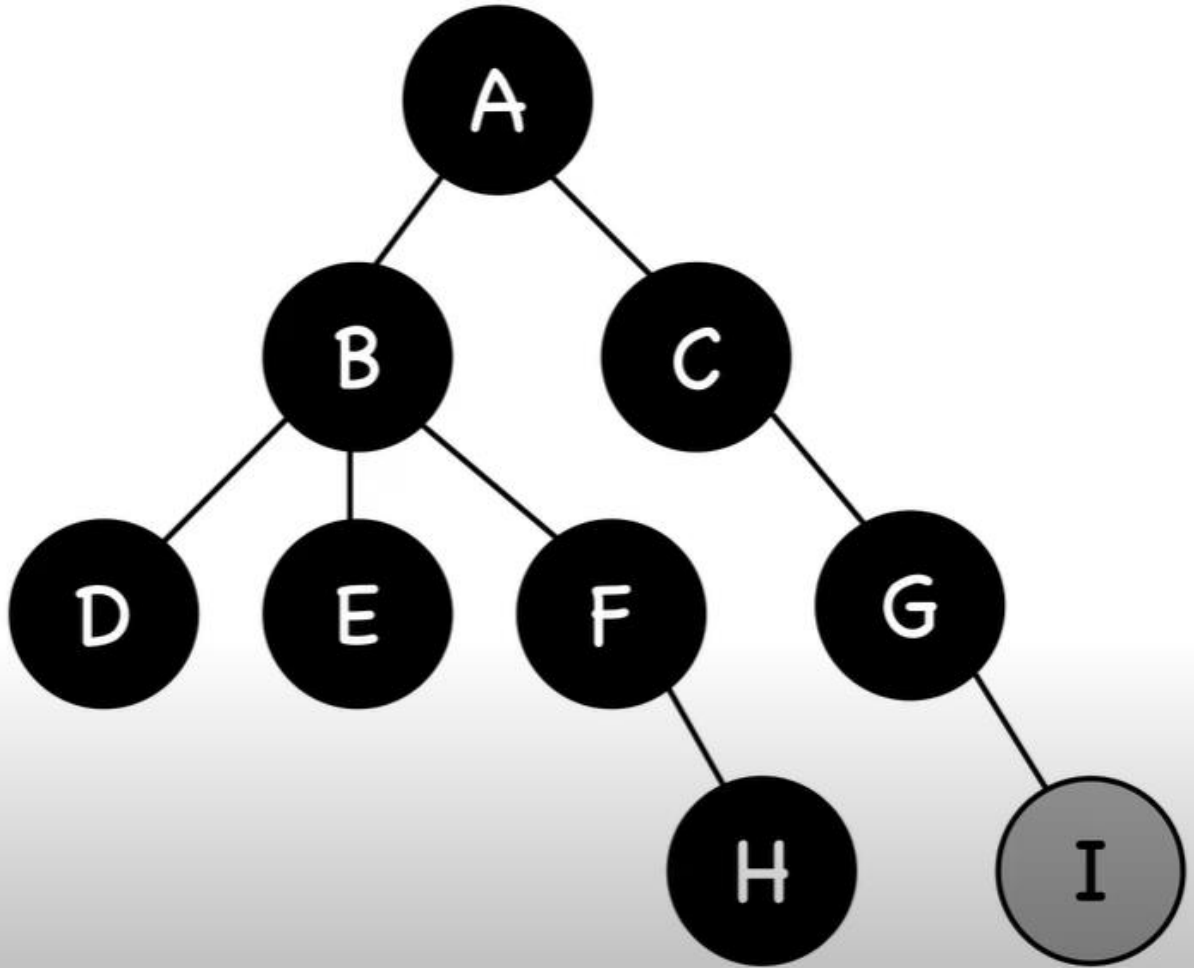


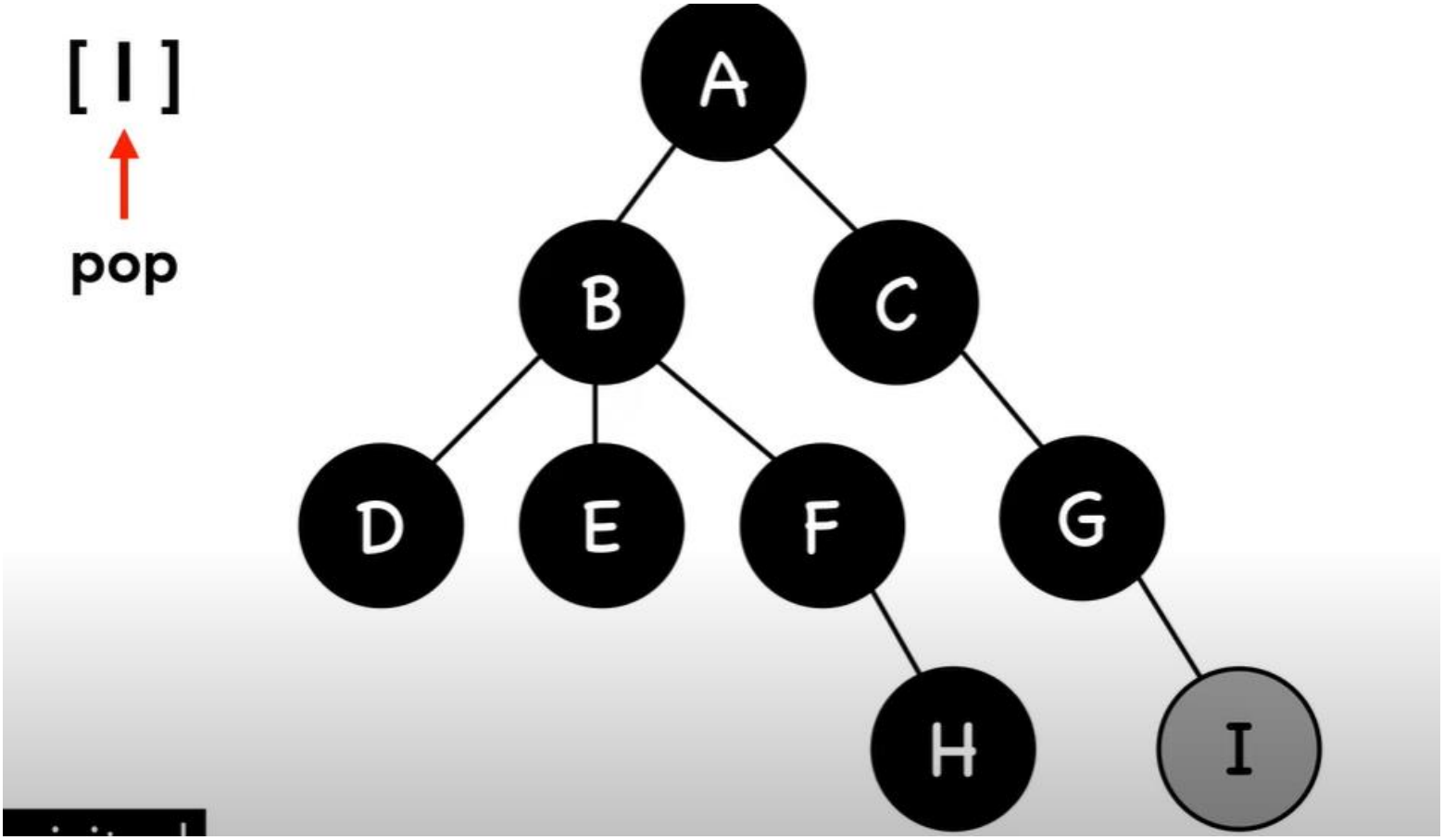
[I, H]

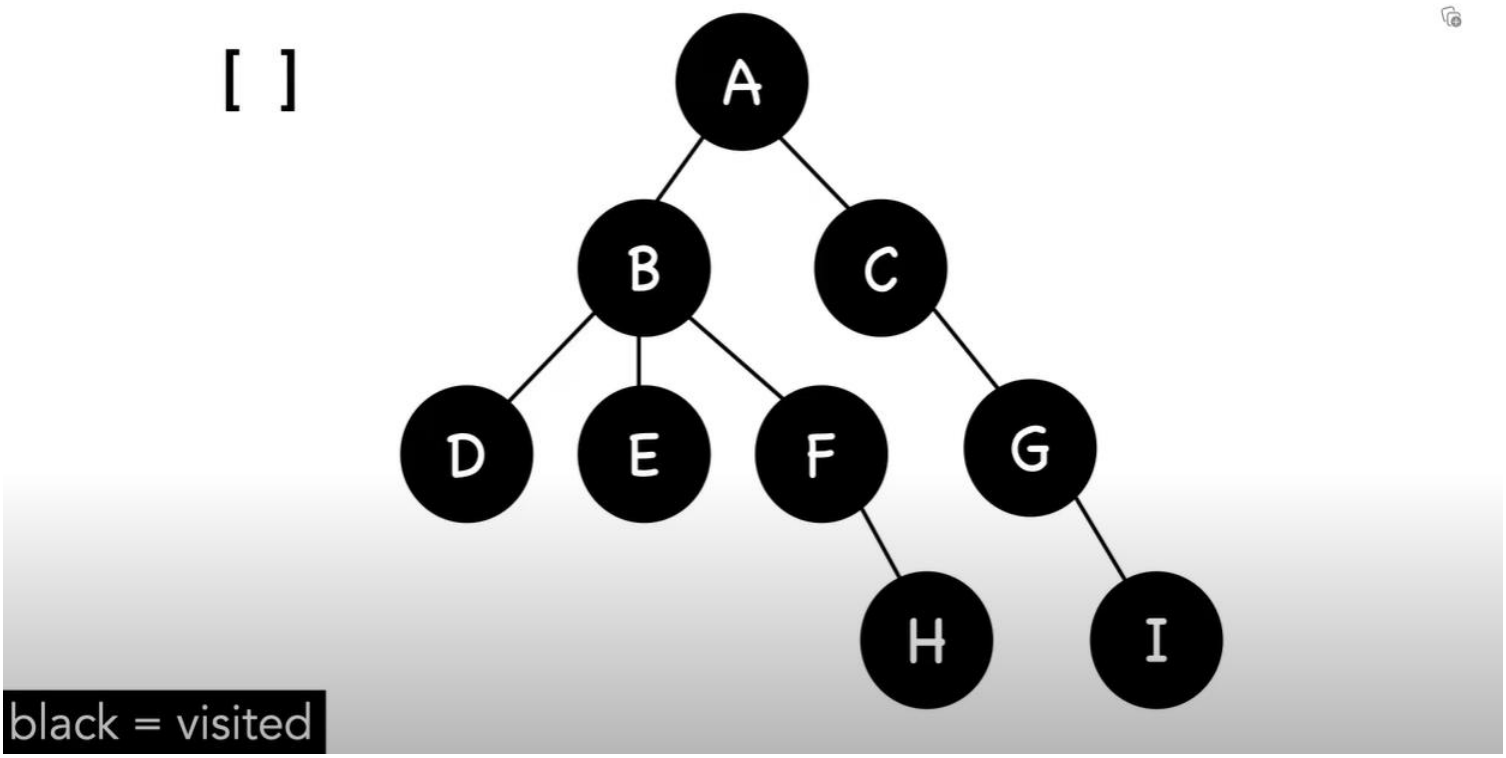



[I, H]
pop
A
B
C
D
E
F
G
H
I

[I]




[I]
pop
A
B
C
D
E
F
G
H
I


[ ]
A
B
C
D
E
F
G
H
I
black = visited

Code of BFS is given below.

```
graph = {
  'A' : ['B','C'],
  'B' : ['D', 'E', 'F'],
  'C' : ['G'],
  'D' : [],
  'E' : [],
  'F' : ['H'],
  'G' : ['I'],
  'H' : [],
  'I' : [],
}
```

```
def bfs(graph, node):
    visited = []
    queue = []

    visited.append(node)
    queue.append(node)

    while queue:
        s = queue.pop(0)
        print(s, end = " ")

        for n in graph[s]:
            if n not in visited:
                visited.append(n)
                queue.append(n)
```

- Explore another algorithm known as Depth-First Search (DFS) using the internet. Then, write code for DFS (Show your visualization work on sheet)